

JSON Schema — API Response Validation

A simple, interview-friendly guide to understanding JSON Schema and using it in API automation.

1. What is JSON Schema?

JSON Schema is a blueprint or rulebook for JSON data. It defines the expected structure, data types, required fields and constraints of a JSON API response.

The API response contains the actual data, while the schema describes what that data is supposed to look like.

2. Example API Response

```
{
  "id": 101,
  "name": "Swapnil",
  "email": "swapnil@test.com",
  "active": true
}
```

3. JSON Schema for the Response

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",

  "type": "object",

  "required": [
    "id",
    "name",
    "email",
    "active"
  ],

  "properties": {
    "id": {
      "type": "integer"
    },
    "name": {
      "type": "string"
    },
    "email": {
      "type": "string",
      "format": "email"
    },
    "active": {
      "type": "boolean"
    }
  }
}
```

4. Understanding Each Part

Schema element	Meaning
"type": "object"	The complete API response must be a JSON object.
properties	Defines the fields that can exist in the JSON object.
"id": {"type": "integer"}	id must contain a whole number.
"name": {"type": "string"}	name must contain text.

Schema element	Meaning
"email": {"type": "string", "format": "email"}	email must be a string following an email format.
"active": {"type": "boolean"}	active must be true or false.
required	Specifies fields that must be present in the response.

5. Properties vs Required

properties describes a field and its rules. **required** says that the field must actually exist.

For example, if email is in properties but not required, the response may omit email. If email is also listed under required, the response must contain email.

6. Valid vs Invalid Data

Valid:

```
"id": 101
```

Invalid because id is expected to be an integer:

```
"id": "101"
```

Similarly, **true** is a boolean, while **"true"** is a string.

7. Why Use JSON Schema in API Automation?

Schema validation is useful when you want to validate the response structure and data types without hardcoding dynamic values such as generated IDs, timestamps or transaction numbers.

8. Rest Assured Example

```
given()
    .when()
    .get("/users/101")
    .then()
    .assertThat()
    .body(matchesJsonSchemaInClasspath("user-schema.json"));
```

Typical project structure:

```
src
├── test
│   ├── java
│   │   └── ...
│   ├── resources
│   │   └── user-schema.json
```

9. Interview-Ready Answer

“JSON Schema is a contract that defines the expected structure, data types, required fields and constraints of a JSON API response. In API automation, I can validate the actual response against this schema to ensure that fields are present and their data types and formats are correct, without necessarily hardcoding dynamic values.”

10. Easy Way to Remember

API Response = Actual data

JSON Schema = Rules for that data

Example:

id	→ integer
name	→ string
email	→ email format
active	→ boolean
required	→ field must exist